

Festival

Nayra está en un festival, participando en un juego cuyo máximo premio es un viaje a Laguna Colorada. El juego consiste en usar fichas para comprar cupones. Comprar un cupón puede hacer que Nayra reciba más fichas. El objetivo es obtener la mayor cantidad de cupones posibles.

Nayra comienza el juego con A fichas. Hay N cupones, numerados de 0 a $N - 1$. Nayra tiene que pagar $P[i]$ fichas para comprar el cupón i (y debe tener al menos $P[i]$ fichas antes de la compra). Cada cupón se puede comprar a lo sumo una vez.

Además, el cupón número i ($0 \leq i < N$) es de **tipo** $T[i]$, que es un número entero **entre 1 y 4, inclusive**. Cuando Nayra compra el cupón i , el número de fichas que le quedan se multiplica por $T[i]$.

Formalmente, si en algún momento del juego tiene X fichas y compra el cupón i (por lo cual, necesariamente $X \geq P[i]$), entonces tendrá $(X - P[i]) \cdot T[i]$ fichas después de la compra.

Tu tarea es determinar qué cupones debe comprar Nayra y en qué orden, para maximizar el número total de **cupones** que tiene al final. Si existe más de una secuencia de compras que lo consiga, puedes informar cualquiera de ellas.

Detalles de implementación

Debes implementar la siguiente función.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : el número inicial de fichas que tiene Nayra.
- P : un arreglo de longitud N especificando los precios de los cupones.
- T : un arreglo de longitud N que especifica los tipos de los cupones.
- Esta función se ejecuta exactamente una vez para cada caso de prueba.

La función debe devolver un arreglo R , que especifica las compras de Nayra de la siguiente manera:

- La longitud de R debe ser el número máximo de cupones que puede comprar.
- Los elementos del arreglo son los índices de los cupones que debe comprar, en orden cronológico. O sea, primero compra el cupón $R[0]$, después el cupón $R[1]$ y así

sucesivamente.

- Todos los elementos de R deben ser únicos.

Si Nayra no puede comprar ningún cupón, R debe ser un arreglo vacío.

Restricciones

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ para cada i tal que $0 \leq i < N$.

Subtareas

Subtarea	Puntos	Restricciones adicionales
1	5	$T[i] = 1$ para cada i tal que $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ para cada i tal que $0 \leq i < N$.
3	12	$T[i] \leq 2$ para cada i tal que $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Existe un orden que permite que Nayra compre todos los cupones.
6	16	$(A - P[i]) \cdot T[i] < A$ para cada i tal que $0 \leq i < N$.
7	18	Sin restricciones adicionales.

Ejemplos

Ejemplo 1

Considera la siguiente llamada.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra dispone inicialmente de $A = 13$ fichas. Puede comprar 3 cupones en el orden que se indica a continuación:

Cupón comprado	Precio del cupón	Tipo de cupón	Número de fichas después de la compra
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

En este ejemplo, es imposible que Nayra compre más de 3 cupones, y la secuencia de compras descrita anteriormente es la única forma en la que puede comprar 3. Por lo tanto, la función debe devolver $[2, 3, 0]$.

Ejemplo 2

Considera la siguiente llamada.

```
max_coupons(9, [6, 5], [2, 3])
```

En este ejemplo, Nayra puede comprar ambos cupones en cualquier orden. Por lo tanto, la función debe devolver $[0, 1]$ o $[1, 0]$.

Ejemplo 3

Considera la siguiente llamada.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

En este ejemplo, Nayra tiene una ficha, que no es suficiente para comprar ningún cupón. Por lo tanto, la función debería devolver $[]$ (un arreglo vacío).

Evaluador local

Formato de entrada:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Formato de salida:

```
S  
R[0]  R[1]  ...  R[S-1]
```

Acá, S es la longitud del arreglo R devuelto por `max_coupons`.